DXF データの作り方

最終更新: 2026-08-23（HBN240018）

`geosim/read_dxffile.py` が読める DXF を CAD 側でどう作るか、という運用ルール。 リポジトリ直下の `test.dxf` が最小の実例（2×3×1 m の直方体）。

結論（この 5 つを守れば読める）

#	ルール	理由
1	**面はポリフェイスメッシュ・3DFACE・閉じたポリラインのいずれかで作る**	この 3 つを面として読む。ソリッドや開いた線は読み飛ばす
2	**同じ材料になる面を同じレイヤにまとめる**（レイヤ名は部位が分かる名前でよい）	材料の割り当ては**条件表（xlsx）**で決めるので、レイヤ名が材料名である必要はない（2026-08-21 に変更。2 節）
3	**法線が「空気側」を向くようにモデルを作る。両面で反射させたいものは厚みを持たせる**	面は**片側だけ反射する**。裏から来た音線は通り抜ける
4	**単位は mm でよい（$INSUNITS が入っていれば自動換算）**	ヘッダの `$INSUNITS` を見て m に直す
5	**音源は `src` レイヤ、受音点は `rec` レイヤに POINT を置く**	座標をコードに書かなくて済む

> **閉じた室である必要はありません。**一面だけの反射板を置いて「直接音と 1 回反射音だけが到来する状況」を検証する、といった使い方ができます（詳しくは 3 節）。
> そのため、**閉じているかを作図ミスとして指摘するのは「閉じたモデルを想定する」と言われたときだけ**にしてある（条件入力の「閉じたモデル」。2026-08-28）。
> どこが開いているかは `python open_edges.py <DXF>` で表・図に出せる。

---

1. 面のつくり方

三角形を手で作る必要はない

計算に使うのは三角形要素だが、**CAD 側で三角形に割っておく必要はない。** 四角形でも多角形でも、読み込み時に自動で三角形へ分割する（n 角形 → n − 2 枚）。

面として読めるのは次の 3 つ。

CAD での描き方	DXF エンティティ	頂点数	例
ポリフェイスメッシュ	`POLYLINE`（`70` に 64）	面レコードごとに 3〜4	`test.dxf`
3D 面	`3DFACE`	3〜4	—
閉じたポリラインで輪郭を描く	`POLYLINE`（`70` に 1）／`LWPOLYLINE`（`70` に 1）	**いくつでも**	`test2.dxf`（9 角形）

- **ポリフェイスメッシュ**：頂点 VERTEX（`70=192`）が並び、そのあとに面レコード VERTEX（`70=128`）が続く。

  面レコードの `71`〜`74` が頂点番号（1 始まり）で、4 番目が 0 なら三角形。
- **3DFACE**：単体で 3〜4 頂点の面を表す。4 点目が 3 点目と同じなら三角形。
- **閉じたポリライン**：面の輪郭を描くだけの作り方。**閉じていれば面とみなして分割する。**

  CAD で作図するのがいちばん楽なのでこれが実用的。**頂点数の制限はない。**
  ※ `LWPOLYLINE` は押し出し方向が Z 以外だと未対応（読み飛ばして報告する）。

読み込み時に、面がどのエンティティ由来かを報告する。

```
[read_dxffile] 面の元になったエンティティ: {'閉じたポリライン': 2}
```

気をつける点

問題	何が起きるか	対処
ねじれた四角形（4 点が同一平面上にない）	どちらの対角線で切るかで形が変わる	**警告が出る。**次節の作図方法で避ける
凹んだ四角形	素朴な分割だと多角形の外に三角形ができる	**対処済み**（内側を通る対角線を自動で選ぶ）
凹んだ多角形（5 角形以上）	同上	**対処済み**（耳刈り法。凹んでいても正しく分割する）
自己交差した多角形	正しく分割できない	**検出して報告する**（面積が合わないので分かる）

分割が正しくできたかは**面積の照合**で自動チェックしている。多角形の面積（靴ひも公式）と 三角形の合計面積が一致しなければ報告する。

```
[read_dxffile] 三角形分割: 5角形以上を 1 枚耳刈り法で分割（面積の相対誤差 最大 1.61e-16）
```

失敗した場合はこう出る。

```
[read_dxffile] ★三角形分割に失敗した多角形が 1 枚あります
   （自己交差しているか、同一平面上にない可能性）
```

ACIS（REGION / 3DSOLID）しか無い図面はプログラムで直せる

2026-08-28 追加。設計図がそのまま REGION / 3DSOLID で出来ている場合 （実務ではこれが普通)、**AutoCAD が入っていれば変換できる。**

```
python dxf_faces.py "C:\...\階段教室.dxf"
    → 同じ場所に 階段教室_faces.dxf ができる
```

中でやっていること：`accoreconsole`（AutoCAD のコマンドライン版）を呼び、 `EXPLODE` で REGION を輪郭の線に、3DSOLID を面の REGION に分解し、 その線を輪の順に並べ直して**閉じた 3D ポリライン**として書き出す。

- **元のファイルは触らない**（控えを作って作業する）
- **レイヤ名は保たれる**（そのまま条件表に使える）
- **★音源・受音点（`src` / `rec` レイヤの `POINT`）も引き継がれる**（2026-09-11 追加)。

  それまでは面だけを書き出していたので、**変換するたびに音源も受音点も消えていた。**
  CAD 側で点を置いておけば、変換後のファイルをそのまま計算に使える
- 次のものは**数えて知らせる**（黙って捨てない）
  - 円弧・スプラインで囲まれた面（直線だけの面しか作れない）
  - **穴のある面**（外周だけを面にするので、開口が塞がっているかもしれない）
  - 閉じなかった輪郭

AutoCAD が入っていない端末では従来どおり、CAD 側で面に分解して書き出す。

結論：平面の閉じたポリラインで輪郭を描くのがいちばん楽。凹んでいても構わない。 気をつけるのは「平面であること」と「自己交差しないこと」の 2 点だけ。

1b. ねじれない面を AutoCAD でどう作るか

「面エンティティにすれば直る」わけではない

まず誤解を解いておくと、**エンティティの種類を面（3DFACE やポリフェイスメッシュ）に変えても ねじれは直らない。**どちらも頂点ごとに z を持てるので、4 頂点なら同じようにねじれる。

効くのは**構造的に平面が保証される描き方をすること。**以下、保証の強い順。

作図方法	平面性	理由
三角形で作る	**原理的に必ず平面**	3 点は必ず 1 つの平面を張る
UCS を作業面に合わせて `PLINE`（LWPOLYLINE）	**構造的に必ず平面**	頂点が xy のみ、高さはエンティティ全体で 1 つ（グループコード 38）。ねじれる余地がない
直線の辺を押し出す（`EXTRUDE` / 立ち上げ）	**必ず平面**	直線 + 平行移動は必ず 1 つの平面を張る
平面で描いてから `3DROTATE` / `ALIGN` で配置	保たれる	剛体変換は平面性を壊さない
`3DPOLY` / `3DFACE` で 3D 空間に直接描く	**ねじれうる**	頂点ごとに z を持てる。`test2.dxf` はこの形式

具体的な手順

(a) 傾いた面 → UCS を合わせてから `PLINE`

```
UCS → 3P（3点指定）で作業面を決める    ← または既存面に UCS → OB（オブジェクト）
PLINE で閉じた輪郭を描く                ← LWPOLYLINE になるので平面が保証される
UCS → W（ワールドに戻す）
```

これがいちばん確実。**LWPOLYLINE は高さを 1 つしか持てないので、ねじれることが原理的にない。**

(b) 壁の立ち上げ → 押し出し

`test2.dxf` の壁の作り方がまさにこれで、**直線の辺を鉛直に押し上げているので必ず平面**になる。 斜め方向に押し出しても平面のまま（検証済み：ねじれ 0.000000000 mm）。

(c) 曲面・ねじれた面 → 三角形で割る

どうしても 4 点が同一平面に乗らない面（ねじれた天井、双曲放物面など）は、 **CAD 側で三角形に割る。** 三角形なら原理的にねじれない。

AutoCAD 内でねじれを検算する裏技

`REGION` コマンドを試す。 REGION は定義上「平面領域」なので、 **平面でない閉じた輪郭に対しては失敗する。**つまり平面性チェックとして使える。

```
閉じたポリラインを選んで REGION → 成功すれば平面、失敗すれば平面でない
（確認したら REGION を削除してポリラインのまま書き出す。REGION は本ツールでは読めない）
```

`UCS → OB`（オブジェクト）でポリラインを指定する方法も同様に、平面でないと通らない。

どのくらいのねじれなら気にしなくてよいか

読み込み時にねじれ量を**実寸（mm）**で報告する。判断の目安はこう。

```
[read_dxffile] 三角形分割: ねじれた四角形 1 枚（4 点が同一平面上にない）
  最大ねじれ 5.000 mm（相対 1.39e-03）… 音響計算に影響しうる大きさです
```

1 m 角の面がねじれたときの影響（法線の傾き → 反射音線のずれ）：

ねじれ	法線の傾き	10 m 先での音線のずれ	判断
0.05 mm	0.003°	1 mm	**無視できる**（スナップの丸め程度）
0.5 mm	0.029°	10 mm	**無視できる**
5 mm	0.29°	100 mm	受音球半径と同程度。**気になるなら直す**
50 mm	2.9°	1000 mm	**作図ミスの可能性が高い。直すべき**

受音球の半径は通常 50〜300 mm なので、**ねじれ 1 mm 程度なら音響的には無視できる。** 警告が出ても値を見て判断すればよく、0.01 mm のずれで作り直す必要はない。

例：test2.dxf（閉じたポリラインで平面 9 角形＋壁）

`test2.dxf` は CAD 上で 2 面だけ作ったもの。

- 閉じた 3D ポリライン 9 頂点（すべて z=0）→ 平面の**凹んだ 9 角形** → 三角形 **7 枚**
- 閉じた 3D ポリライン 4 頂点 → 9 角形の 1 辺を z=1500 まで立ち上げた**垂直な壁** → 三角形 **2 枚**

面積の照合結果は 0.344704277 m²（靴ひも公式）に対し相対誤差 1.6e-16 で一致。

法線は、9 角形が反時計回りに描かれているので**床が上向き（+z）**、壁は **9 角形の内側（床の上の空間）を向く**。どちらも空気側なので正しい。

1 つのレイヤに複数の POLYLINE を置いてよい

`test.dxf` では壁 4 面がすべてレイヤ `2` にまとまっている。 **吸音材を変えたい面だけレイヤを分ける**運用がいちばん楽。

```
レイヤ '1' … 床       （POLYLINE 1 枚 → 三角形 2 枚）
レイヤ '2' … 壁 4 面  （POLYLINE 4 枚 → 三角形 8 枚）
レイヤ '3' … 天井     （POLYLINE 1 枚 → 三角形 2 枚）
```

避けること

- **面積ゼロの面（縮退面）**。3 点が一直線に並んだ三角形。

  法線が計算できないので自動で捨てるが、意図せぬ穴になるので CAD 側で作らない。
- **面の重複**。同じ場所に 2 枚あると音線が余分に反射する。

  読み込み後に出る表面積を検算するとよい（後述）。
- **ねじれた四角形**（上記）。

2. レイヤ名 = 吸音材の「区分」

★★2026-08-21 に考え方が変わった。CAD 側は「壁を区分ごとにレイヤで分ける」だけでよく、 レイヤ名が材料名である必要はない。材料の割り当ては **条件表**（`プロジェクトフォルダ/条件表.xlsx`）で決める。

> 依頼（2026-08-21）:「壁の吸音材条件を変える場合に、CAD データを変えて、
> それをまた読み込んでもらって、っていうのが大変な気がします。」

つまり CAD 側の役割はこうなった。

	CAD 側でやること	条件表でやること
いま	**同じ材料になる面を同じレイヤにまとめる**（レイヤ名は分かる名前でよい）	レイヤーごとに**材料番号**と**安全率**を書く
以前	レイヤ名を材料名（または吸音率 CSV の ID）と一致させる	—

- レイヤ名は `01__研修室_壁_扉` のように**部位が分かる名前**にしておけばよい
- **条件（吸音材の組み合わせ）はシートで分ける。**1 シート = 1 条件で、

  シート名が条件名になり結果ファイル名に入る。一括で回して比較表も作れる
- 条件表は設定画面の「**条件表を作成**」で、いま読み込んでいる DXF のレイヤを

  並べた雛形を作れる（利用者が書いた材料番号は**絶対に上書きしない**）
- 詳しくは `docs/技術説明書.md` 9.5.0 節

吸音率の一覧（材料番号 → α）

条件表の「**吸音率**」**シート**に、PJ で使う材料を並べる（番号・材料名・α・**種類**）。 **番号は行の位置**（`=ROW()-1`）で、条件シートからはこの番号で引く。 ★材料名を入力に使わないのは、全角・半角のミスタイプを誘発するため（ユーザー指摘）。

吸音率 CSV を使うこともできる（条件表の「吸音率」シートが無いとき）。 CSV は 2 つの形式を自動判別する。

(A) 元コード付属の `absorption.csv` 形式（ID + 材料名 + a1〜a6）

```
1,Concrete  wall,0.01,0.02,0.02,0.02,0.03,0.04
2,Cloth  on  concrete  wall,0.03,0.03,0.03,0.04,0.06,0.08
3,Needle  punch,0.03,0.04,0.06,0.1,0.2,0.35
```

ID と材料名の両方をキーに登録するので、DXF のレイヤ名が `1` でも `Concrete wall` でも引ける。`test.dxf` はレイヤ名が `1` / `2` / `3` なのでそのまま使える。

(B) 材料名 + a1〜a6

```
コンクリート,0.02,0.02,0.03,0.03,0.04,0.05
グラスウール25mm,0.15,0.35,0.65,0.85,0.90,0.85
```

- **数値 ID でも材料名でもよい**。後から見て分かるのは材料名なので、**材料名を推奨**。
- 日本語のレイヤ名・材料名も読める

  （エンコーディングは DXF が UTF-8 → CP932 → latin-1、CSV が UTF-8 → CP932 → latin-1 の順に試す）
- CSV に無いレイヤがあると**警告が出て既定値 0.1 が使われる**。警告は無視しないこと。

サンプル: `data/absorption_sample.csv`（形式 B）／元コード付属のものは `absorption.csv`（形式 A）

> ⚠ **入れるのは垂直入射（法線入射）吸音率**。残響室法（乱入射）吸音率は 1 を超えることがあり、`sqrt(1 - α)` が NaN になって計算が壊れる。詳細は `TODO.md` の D-7、`docs/技術説明書.md` 7.3 節。

レイヤで分けられないモデル（1 つの 3DSOLID など）

他ソフトの都合で **1 つのソリッドとして作らざるを得ない**ことがある。 このとき面ごとのレイヤが無いので、上の突き合わせが使えない。 その場合は **`face_editor.py`（面の確認ウィンドウ）で面を選んで直接貼る**。

- 同一平面で連結した三角形は**自動的に 1 枚の面としてまとめられる**ので、

  三角形を 1 枚ずつ選ぶ必要はない（ModelTest は 68 三角形 → 16 面）
- `k` で**同じ平面**の面を、`t` で**同じ平面かつ同じレイヤ**の面をまとめて選べる

  （2026-08-21 に「同じ向き」から「同じ平面」に変えた。向きだと離れた壁まで拾う）
- 面は**同一平面パッチを 1 枚として見せる**（三角形の割れ目は描かず、

  パッチの外周だけを線で重ねる）
- ★**「選ぶ → 適用する」の二段階**。枠で囲んだ瞬間に変わるのではなく、

  選択が橙で残り、キーで適用する
- 結果は `materials.json` に保存され、次からは自動で適用される

  （**DXF 名と面数を控えてあり、食い違ったら使わずに知らせる**）

- **CAD 側でレイヤを分けられるなら、そちらのほうがよい。**

  面ごとの割り当ては DXF の面の並び順に依存するので、モデルを描き直すとやり直しになる

DWG（ソリッド）を読ませるには

`3DSOLID` は ACIS の境界表現なので、このプログラムでは直接読めない（`read_dxffile` は面 — `3DFACE` / ポリフェイスメッシュ / 閉じたポリライン — だけを読む)。 AutoCAD で三角形の面に変換してから読ませる。2026-08-19 に `ModelTest.dwg` で確認した手順:

1. `accoreconsole.exe /i model.dwg /s script.scr` で `STLOUT` を実行し、ASCII STL に出す

   （`CONVTOMESH` は accoreconsole には無い）
2. STL の三角形を `3DFACE` として書き直し、元 DXF から `$INSUNITS` と

   `src` / `rec` の `POINT` を引き継ぐ
3. 変換が正しいかは **AutoCAD の `MASSPROP` の体積**と読み込み後の体積を突き合わせる

   （面が全部平面なら誤差なしで一致する）

---

3. 法線の向き — モデル側で正しく作る

大原則：法線は「音が通る空気側」を向く

音線が壁に当たったと判定されるのは `v·n < 0`（**法線に向かって進んでいる**） のときだけ。 つまり**法線の裏側から来た音線は、その面を通り抜ける**（面は片側だけ反射する)。

したがって作図のルールはこう決まる。

> **反射させたい空気の側に法線を向ける。**

- **室の外殻** … 法線を室内向きに（空気は内側にある）
- **室内に置く物体（机・什器など）** … 法線を外向きに（空気は物体の外側にある）
- **一面だけの反射板** … 反射させたい側に法線を向ける

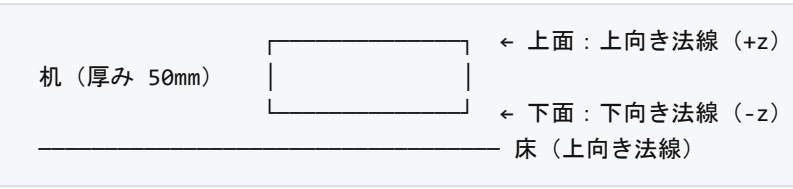
この向きの正解を持っているのは CAD モデル自身なので、読み込み側は既定で **CAD の巻き順をそのまま使う**（`orient_normals='cad'`)。勝手な補正はしない。

> ⚠ **「法線を音源方向に向ける」のは誤り。**
> 凸凹のある壁や、宙に浮いた家具では向きが破綻して衝突判定がおかしくなる。
> かつて `orient_normals='toward'` という機能を用意していたが、この理由で**廃止した**。

両面で反射させたいものは厚みを持たせる

面は片側だけ反射するので、**1 枚の面で両面反射はできない。** 両側から音が当たる物体は、CAD 側で**厚みのある立体**として作る。

例：室内に置く厚みのある机（脚は省略）



上面は上向き、下面は下向きになり、**どちらも空気側**を向く。 こうすれば机の上を通る音線は上面で反射し、机の下を通る音線は下面で反射する。

薄い板でも同じで、厚みゼロの 1 枚面にすると片側しか反射しない。 両面反射を見込みたいなら厚みを付ける。

読み込み時のシェル診断で向きを確認する

読み込み時に、辺の共有で**シェル（連結した面のかたまり）**に分割し、 シェルごとに閉/開・体積・法線の向きを報告する。ここを見れば向きの誤りに気づける。

室 4×5×3 m ＋ 厚みのある机 1.6×0.8×0.05 m の例:

```
[read_dxffile] シェル（連結した面のかたまり）: 2 個
[read_dxffile]   シェル0 [外殻] 面12枚 閉 体積60.0000m³ 法線=inward  OK 寸法[4.0, 5.0, 3.0]
[read_dxffile]   シェル1 [内側] 面12枚 閉 体積 0.0640m³ 法線=outward OK 寸法[1.6, 0.8, 0.05]
```

- **外殻**（他のすべてを内包するシェル）は `inward` が空気側 → OK
- **内側**のシェル（机など）は `outward` が空気側 → OK
- 逆になっていると `★要確認（outward が空気側）` のように出る

巻き順が一貫していない場合（隣り合う面で法線が反対を向いている場合）も検出して警告する。 これは**同じ有向辺が 2 回現れるか**で判定できる （巻き順が揃った面同士が辺を共有するとき、その辺は互いに逆向きに現れるはず）。

`orient_normals` の選び方

値	挙動	使う場面
`'cad'`（既定）	CAD の巻き順をそのまま使う	**通常はこれ。**モデルが正解を持っている
`'flip'`	全反転	モデル全体の向きが逆だったとき。**元コード `ynnmrev='y'` と同じ**
`'shells'`	シェル単位で空気側へ揃える（外殻は内向き、内側は外向き）	CAD 側の向きが不揃いなときの救済

`'shells'` は面ごとの向きを勝手にいじるのではなく、**閉じたシェル単位で一括反転**するだけなので、 凸凹の壁でも壊れない（凸部が外殻と一体なら外殻シェルの一部として扱われる）。 ただし巻き順が一貫していないと補正を中止して `'cad'` と同じ挙動になる。

> 元コード（ `backtrace.f90` 276〜282 行）は `ynnmrev` という y/n フラグで
> 全法線を反転するかをユーザーに聞くだけだった。 `'flip'` がそれに相当する。
> シェル診断と `'shells'` は Python 版で追加した機能。

閉じた室でなくてもよい

面の片側性のおかげで、**閉じた形状である必要はない。** 一面だけの反射板を置いて「直接音と 1 回反射音だけが到来する状況」を検証できる （音線は反射後に飛び去り、当たる壁がなくなった時点で追跡が打ち切られる）。

開いた形状のときは、シェル診断が閉じていないことを報告する。

```
[read_dxffile]   シェル0 [外殻] 面2枚 開（開いた辺4本）法線=CAD のまま 寸法[10.0, 10.0, 0.0]
[read_dxffile]   ※開いた形状（一面反射など）も計算できます
```

期待した反射音が出ないときは、その面の法線が反射させたい側と逆を向いている。 CAD 側で直すか、全体が逆なら `orient_normals='flip'` を使う。

GUI で調整する（2026-08-19 に実装済み）

`face_editor.py`**（面の確認ウィンドウ）で、目で見ながら面単位で直せる。** 読み込み時の自動補正に頼らないのが本来の形、という当初の狙いどおりになった。

- 法線の向きと**面ごとの吸音材**の両方をここで直す

  （2026-08-19 に `normal_editor.py` から改名した）
- 裏側が赤く塗られるので向きの誤りが一目で分かる。矢印の表示も切り替えられる
- `i` で選んだ面の法線を反転、 `a` で自動判定どおりに揃える、 `d` で CAD の巻き順に戻す
- 結果は `normals.json` に保存される（ `materials.json` と同じ約束）
- `m` で **法線 → 吸音材 → 容積の拾い方** と表示を切り替えられる。

  「容積の拾い方」は**どの面を拾ってどの領域を容積に数えたか**を色で見るためのもので、
  赤い線が「開いた辺」。穴なのか T 字接合なのかがここで分かる
- **受音点の正面方向**もここでスライダで決める（出力⑤ 伝搬方向の図に使う）

シェル診断の結果は `DxfModel.shells` / `.winding_consistent` に入っている。

---

4. 単位

DXF ヘッダの `$INSUNITS` を見て m に換算する。`test.dxf` は `$INSUNITS = 4`（ミリメートル）。

$INSUNITS	単位	m/単位
0	unitless（判定不能）	—
1	インチ	0.0254
2	フィート	0.3048
4	**ミリメートル**	**0.001**
5	センチメートル	0.01
6	メートル	1.0

（他にキロメートル・ヤード・ミクロンなども対応。`INSUNITS_TO_METER` を参照）

- `$INSUNITS = 0`（**unitless）だと判定できない**ので警告が出て m 扱いになる。

  CAD 側で単位を設定するか、`read_model(..., unit='mm')` と明示指定する。
- 文字列（`'mm'`, `'cm'`, `'m'`, `'km'`, `'inch'`, `'ft'`）でも数値（m/単位）でも指定できる。
- **明示指定はヘッダより優先される**。CAD の設定が信用できないときはこれを使う。

---

5. 音源・受音点も CAD に入れておける

`src` レイヤに POINT を置けば音源、`rec` レイヤなら受音点として読む。`test.dxf` は音源 (1000, 500, 500) mm、受音点 (700, 2000, 500) mm が入っている。

```
model = rd.read_model("test.dxf")
model.source_points    # [array([1.0, 0.5, 0.5])]
model.receiver_points  # [array([0.7, 2.0, 0.5])]
```

`procedure.process()` に `soundsource_point=None, receiver_point=None` を渡すと DXF 側の値が使われるので、**座標をコードに書かなくて済む**。

- レイヤ名は大文字小文字を区別しない。`source` / `receiver` / `音源` / `受音点` も可

  （`source_layers` / `receiver_layers` 引数で変更できる）
- **複数点を置いてもよい**。リストで全部返る。受音点を何箇所も評価したい場合に使える

  （現状のパイプラインは先頭の 1 点だけを使う。複数受音点への対応は今後の課題）

---

6. 読み込めたかの確認方法

`read_model()` は読み取り結果を print する。ここを見れば大半の作図ミスは分かる。

```
[read_dxffile] 三角形メッシュ: 12 枚
[read_dxffile] 単位: 1 CAD単位 = 0.001 m（$INSUNITS=4）
[read_dxffile] レイヤ別の枚数: {'1': 2, '2': 8, '3': 2}
[read_dxffile] 音源: [[1.0, 0.5, 0.5]]
[read_dxffile] 受音点: [[0.7, 2.0, 0.5]]
[read_dxffile] 形状: 閉じている（室容積 6.0000 m³）
[read_dxffile] 寸法: [2.0, 3.0, 1.0] m (min=[0.0, 0.0, 0.0])
[read_dxffile] 法線の向き: 閉じた形状（開いた辺 0 本）/ 符号付き体積 -6.0000 m³ → すでに内向き
```

チェックポイント:

- **寸法**が意図した室の大きさになっているか（単位換算のミスがここで出る）
- **形状**が意図どおり閉／開になっているか。

  閉じているつもりなら**室容積**が手計算と一致するか（面の抜けがあると合わない）
- **レイヤ別の枚数**が想定どおりか（レイヤの割り当て漏れがここで出る）
- **三角形の枚数**（四角形 1 枚 = 三角形 2 枚）
- **読み飛ばし**の行が出ていないか（縮退面・面レコード不正）
- **吸音率が未指定のレイヤ**の警告が出ていないか
- 開いた形状の場合、**期待した反射音が出ているか**（出ていなければ法線の向き。3 節参照）

さらに**表面積を検算**すると分割の健全性まで確認できる。

```
area = sum(0.5 * np.linalg.norm(np.cross(m.vertexes[1] - m.vertexes[0],
                                         m.vertexes[2] - m.vertexes[0]))
           for m in model.mesh)
# test.dxf → 22.0000 m²（= 2×(2×3 + 3×1 + 1×2)）ぴったり一致すれば重複・欠落なし
```

済んだこと（かつての「今後の課題」）

- **複数受音点** … 対応済み。`rec` レイヤの点を全部使い、結果は `結果/recN/` に分かれる。

  音線追跡は**受音点に依らないので 1 回で済ませる**（受音球を複数置いて同時に判定）
- **作図ミスの自動チェック** … `check_model()` にまとめた

  （閉じているか／自由端／三角形の形／レイヤと吸音率の突き合わせ／容積の検算）
- **受音点の頭部向き** … CAD では表さず、**3D 画面のスライダ**で決める

（`face_editor`。受音点ごとに持てる）

残っている課題

- **元の DXF（REGION / 3DSOLID）を直接読む**（TODO B-22）。

  いまは AutoCAD 側で面（3DFACE / POLYLINE メッシュ）に変換してから読ませる
- **音源の指向性**。いまは点音源（無指向）だけ
- **ブロック参照（INSERT）の展開**。現状は読み飛ばす